

Cross-Site Scripting (XSS)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un XSS ocurre cuando la app mete tu entrada dentro del HTML de una pagina sin neutralizar caracteres especiales (<, >, "), y el navegador de OTRA persona (o el tuyo) interpreta esa entrada como código HTML/JavaScript en vez de texto. Hay tres tipos: reflejado (el payload viaja en la URL/peticion y se refleja en la respuesta inmediata), almacenado (el payload se guarda en la BD y se sirve a cualquiera que vea esa pagina), y DOM-based (el JavaScript del CLIENTE inserta el payload sin pasar por el servidor).

El HTML no distingue 'texto que el usuario escribió' de 'código que el navegador debe ejecutar' salvo que la app escape los caracteres especiales antes de insertarlo. Es la misma causa raíz que `sqli` (mezclar dato y código), aplicada al navegador en vez de a la base de datos.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier campo cuyo valor termine mostrándose en HTML: comentarios, nombres de perfil, búsquedas, mensajes de error que reflejan tu input, parámetros de URL que se pintan en la pagina.

Cómo identificarla

- Tu entrada aparece reflejada tal cual en el HTML de la respuesta (view-source lo confirma).
- Un campo de comentario/perfil que otros usuarios ven, sin indicación de que se sanea.
- El payload de prueba dispara el `alert/console.log`.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Inyecta un payload mínimo y visible, como `<script>alert(1)</script>` o `<img src=x onerror=alert(1)>`, en el campo sospechoso.

Evidencia esperada: Un diálogo de alerta ejecutándose en el navegador (o `console.log` visible en devtools) confirma que tu HTML/JS se interpreto, no se mostro como texto.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/search?q=<script>alert(1)</script>"
```

Objetivo y contexto: Prueba rápida por curl para ver si el payload se refleja SIN escapar en el HTML de respuesta, antes de abrir un navegador.

Resultado esperado: Busca `<script>alert(1)</script>` literal en el HTML devuelto (no `<script>`). Si aparece sin escapar, confirma en un navegador real para ver la ejecución.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/search?q=<script>alert(1)</script>"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un diálogo de alerta ejecutándose en el navegador (o `console.log` visible en devtools) confirma que tu HTML/JS se interpreta, no se muestra como texto.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica donde se refleja o almacena tu entrada viendo el HTML fuente (no solo lo que se ve renderizado).
- Prueba un payload mínimo de detección (`<script>alert(1)</script>`); si se filtra, prueba variantes (atributos de evento, distintas etiquetas).
- Distingue el tipo: si el payload va en la URL y se refleja al momento, es reflejado; si persiste tras recargar o lo ve otro usuario, es almacenado.
- En un CTF/lab, el impacto típico es robar la cookie de sesión de otro usuario o forzar una acción en su nombre.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app escapa los caracteres especiales (`<` se convierte en `<`) antes de insertar tu texto: aparece como texto literal, no se ejecuta.

- Un Content-Security-Policy estricto bloquea la ejecución de scripts inline, incluso si el HTML se inyecta.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada se inserta en el HTML/JS de la página sin escapar; un navegador que la visita ejecuta el JavaScript que tu pusiste. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(xss\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.